Nagios Monitoring Lab

Linux, Windows and Bacula Backup Infrastructure Monitoring

Technical implementation, troubleshooting, operational verification and email alert testing
8 October 2026

Scope: reconstructed from the documented lab sessions and available screenshots. Commands marked “example” should be compared with live configuration before reuse. No secrets are included.

1. Project overview

A multi-host monitoring environment built around Nagios Core 4.4.6 on a Debian virtual machine. It monitors an HP Z400 Linux workstation, a physical Windows 10 desktop and a Debian Bacula backup server, including the health and age of scheduled backup jobs. NRPE, NSClient++, Nagios plugins, Apache CGI, Exim4 and RRDTool-based graphs were used.

2. Lab topology and inventory

Node	IP address	Role
nagios-server	X.X.X.83	Debian VM, Nagios Core, Apache, Exim4, graphs
HP Z400 / z400	X.X.X.201	Linux host, NRPE, SSH, CPU/RAM/disk checks
Windows 10 / windows-physical	X.X.X.184	Physical desktop, NSClient++ NRPE checks
bacula-server	X.X.X.118	Debian Bacula services, PostgreSQL, NRPE
Synology NAS	X.X.X.145	NFS storage used by Bacula: /volume1/BaculaBackups

Monitoring path: Nagios Server → NRPE or NSClient++ → host checks. For backup health: Nagios Server → NRPE on Bacula Server → PostgreSQL Bacula catalogue. The NAS is checked through the Bacula server’s /backup mount.

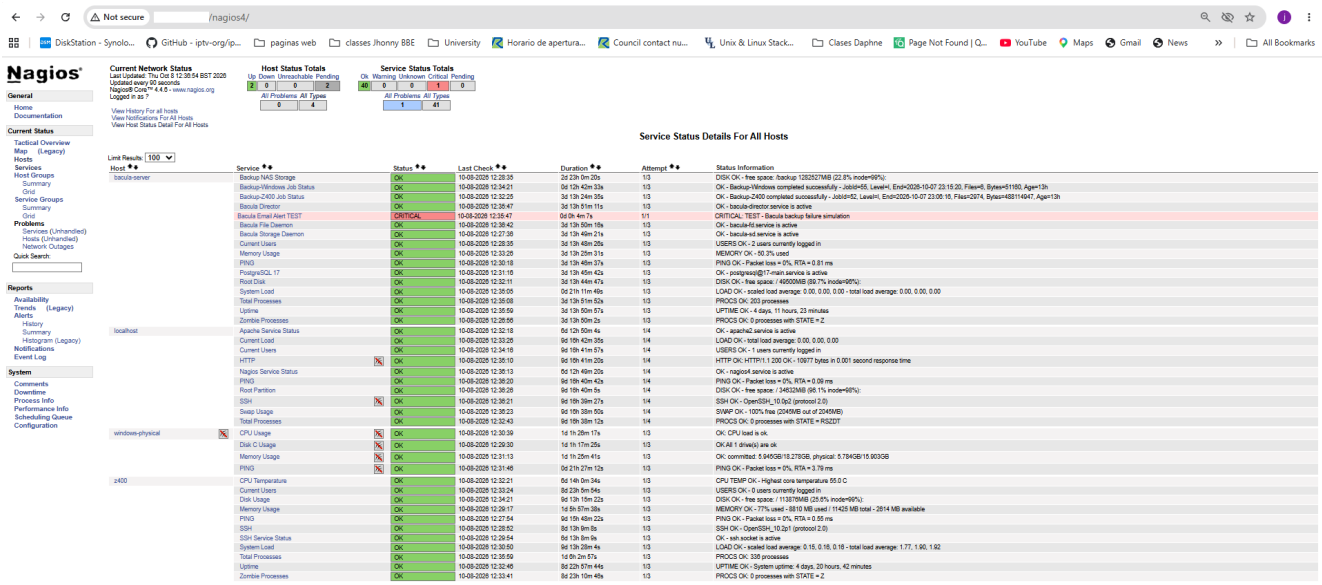


Figure 1 — Nagios Services overview during the controlled CRITICAL email test (the red test service was subsequently removed).

3. Install and verify the Nagios server

Run on NAGIOS-SERVER (X.X.X.83). Debian package installation and validation commands:

```
sudo apt update
sudo apt install nagios4 nagios-plugins apache2
sudo systemctl enable --now nagios4 apache2
sudo nagios4 -v /etc/nagios4/nagios.cfg
systemctl status nagios4 --no-pager
```

The web UI is available at http://X.X.X.83/nagios4/. The validated installation reported Nagios Core 4.4.6. The final pre-flight check before the temporary alert test reported 39 services, four hosts, 216 commands, zero warnings and zero errors; the Windows backup service then brought the production total to 40 services.

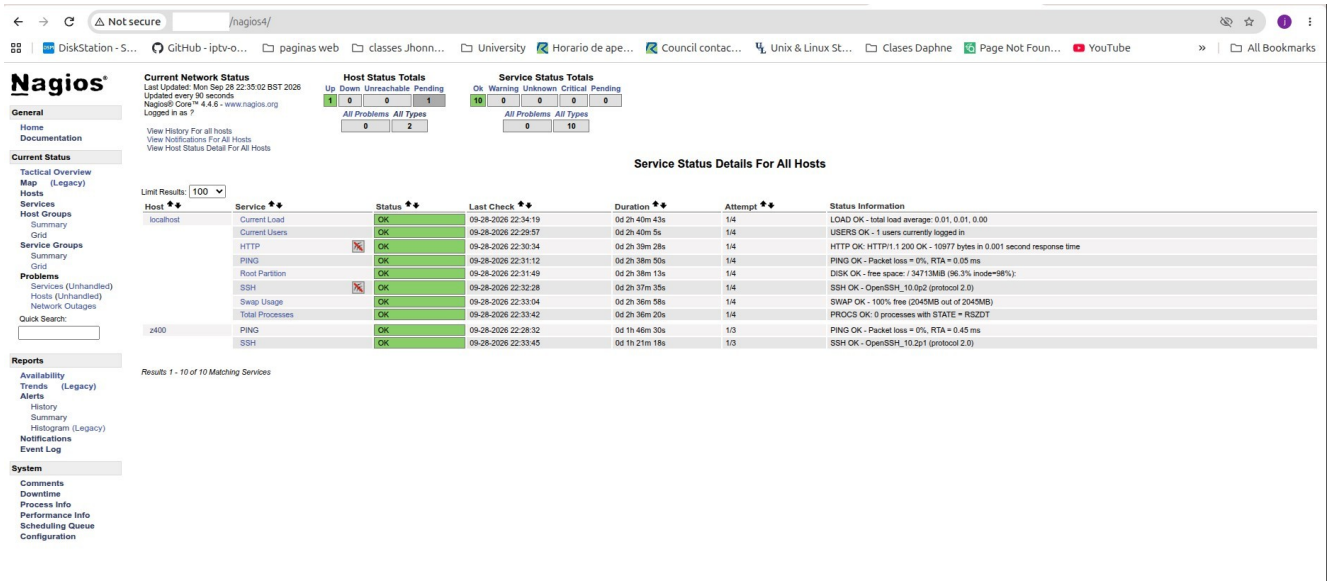


Figure 2 — Early Nagios Services view following initial host configuration.

4. Apache CGI troubleshooting

Problem: selecting Hosts/Services caused the browser to download status.cgi rather than execute it. Apache CGI support and ScriptAlias paths were reviewed. Run on NAGIOS-SERVER:

```
sudo a2enmod cgi
sudo systemctl restart apache2
/usr/sbin/apache2ctl -M | grep cgi
curl -I http://localhost/nagios4/cgi-bin/status.cgi
```

Verified CGI executable: /usr/lib/cgi-bin/nagios4/status.cgi. Apache mapping: /etc/apache2/conf-enabled/nagios4-cgi.conf, with /cgi-bin/nagios4 and /nagios4/cgi-bin aliases. The Nagios UI CGI base URL was corrected in /usr/share/nagios4/htdocs/config.inc.php to /nagios4/cgi-bin. A local HTTP 200 response and functioning Hosts/Services pages confirmed resolution.

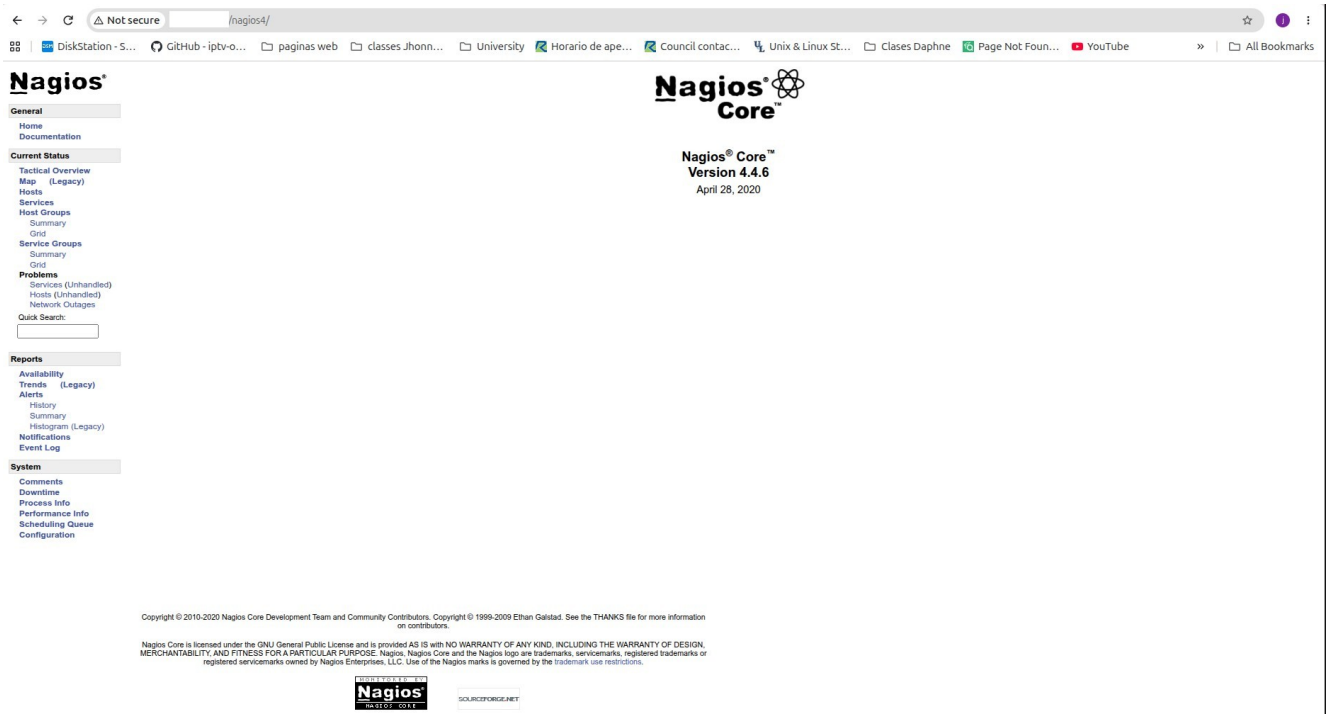


Figure 3 — Nagios web interface during initial CGI configuration and testing.

5. Linux monitoring — HP Z400

On the HP Z400 (X.X.X.201), NRPE 4.1.3 and OpenSSH were used. Nagios checks included PING, SSH, system load, memory, root disk, processes, zombies, logged-in users, uptime, CPU temperature and SSH socket state. Configure NRPE to accept requests from the Nagios server, with the least restrictive access necessary.

```
# NAGIOS-SERVER
/usr/lib/nagios/plugins/check_ssh -H X.X.X.201
/usr/lib/nagios/plugins/check_nrpe -H X.X.X.201
# HP Z400
systemctl status ssh.socket --no-pager
nproc
lscpu
```

NRPE command configuration was held in /etc/nagios/nrpe.cfg on the Z400. Custom checks included check_memory, check_uptime, check_cpu_temp and check_systemd_service under /usr/lib/nagios/plugins/. Initial load thresholds were far too low for the 12 logical CPU system; thresholds were revised after checking nproc and lscpu. A later load check returned OK (0.95, 0.84, 0.81).

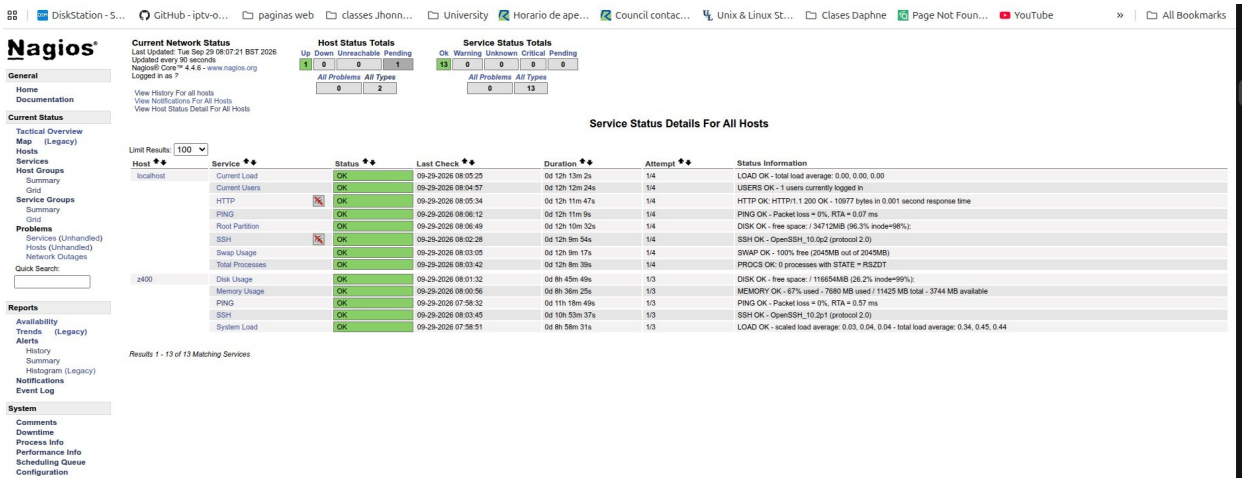


Figure 4 — Linux services reporting OK in Nagios.

6. Windows monitoring — NSClient++

The physical Windows 10 host at X.X.X.184 was monitored through NSClient++ with NRPE-compatible CPU, memory and disk checks plus PING. Allowed hosts were limited to the Nagios server; NSClient++ NRPE TLS settings were adjusted for interoperability (verify mode = none). This is a lab compatibility setting, not a hardened production recommendation.

The Windows machine can sleep. To prevent repeated false-positive email alerts, notifications_enabled 0 was applied to the Windows host and four Windows services, while monitoring remained active. Restore notifications only after addressing sleep behaviour and the desired maintenance schedule.

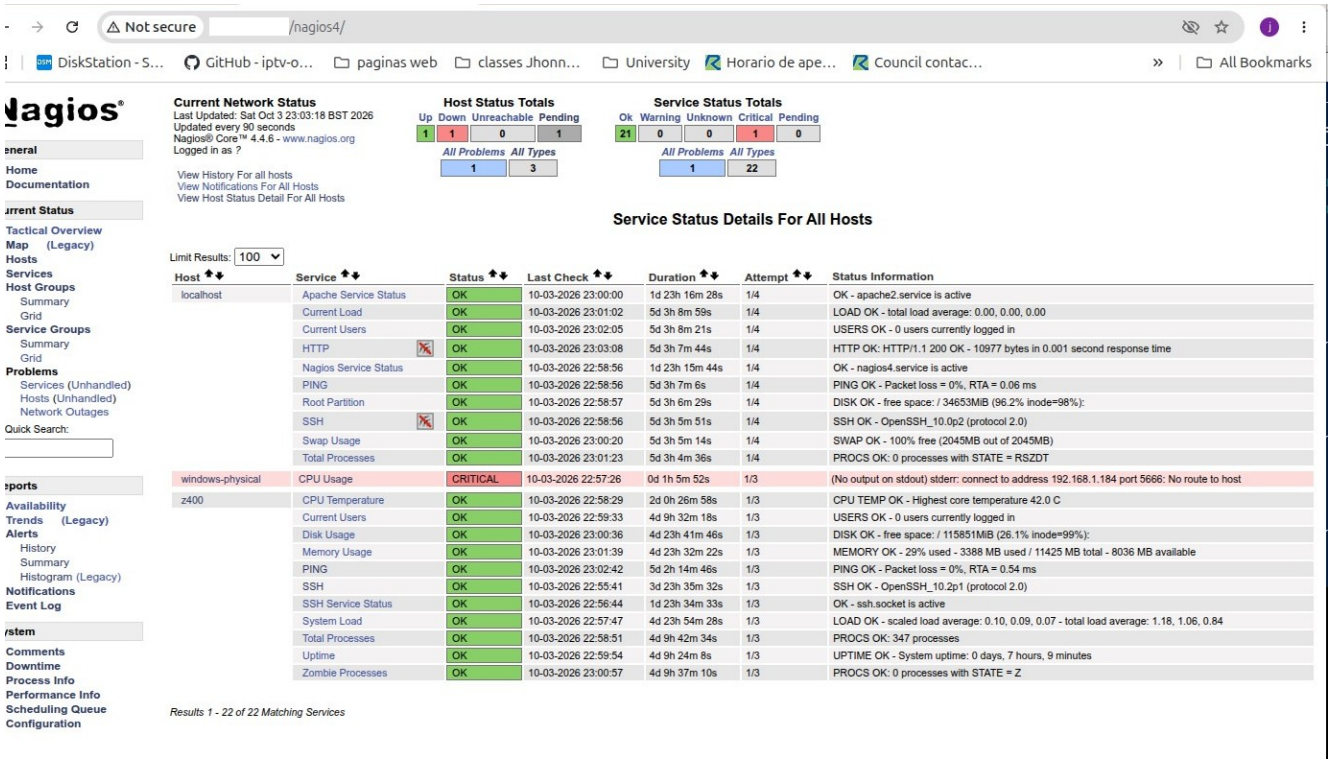


Figure 5 — Windows and other monitored services during testing; intermittent Windows reachability was investigated.

7. Bacula server and NAS checks

On BACULA-SERVER (X.X.X.118), NRPE exposes system load, memory, disk, uptime, users, process and zombie counts, Bacula Director/Storage/File Daemon service health, PostgreSQL 17, and NAS mount health at /backup. The Nagios host definition is stored in /etc/nagios4/conf.d/bacula-server.cfg.

```
# BACULA-SERVER
df -h /backup
systemctl status bacula-director bacula-sd bacula-fd --no-pager
# NAGIOS-SERVER
/usr/lib/nagios/plugins/check_nrpe -H X.X.X.118
```

A 5.4 TB NFS-backed /backup mount was confirmed in the Bacula lab. Do not assume a service is healthy merely because the NRPE daemon responds; check each service result in Nagios.

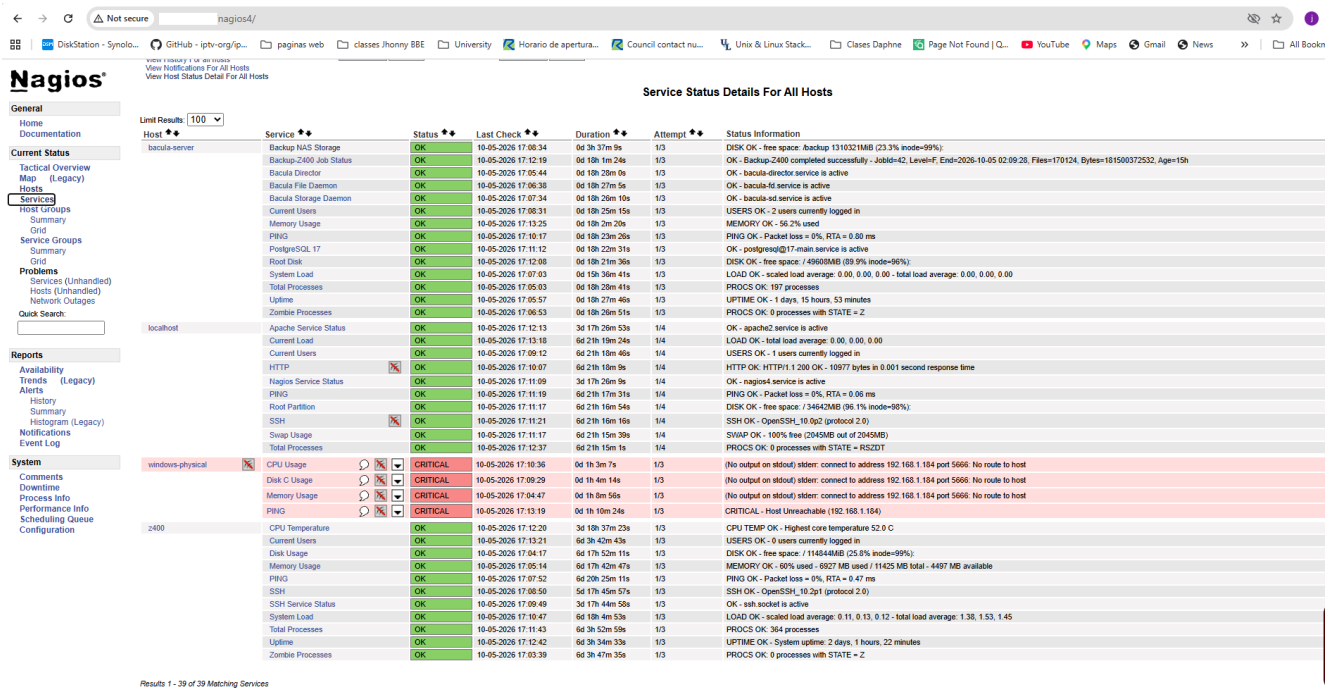


Figure 6 — Bacula server monitoring and backup-status checks in the Nagios Services page.

8. Monitor the Bacula backup jobs

Two independent scripts on BACULA-SERVER query the Bacula PostgreSQL catalogue using a least-privilege database role. The Z400 script is /usr/lib/nagios/plugins/check_bacula_backup and the Windows copy is /usr/lib/nagios/plugins/check_bacula_windows_backup. Both use 30-hour WARNING and 48-hour CRITICAL thresholds; they also return CRITICAL if the latest completed job failed. A running job returns OK.

```
# BACULA-SERVER: /etc/nagios/nrpe.cfg
command[check_bacula_backup]=/usr/lib/nagios/plugins/check_bacula_backup
command[check_bacula_windows_backup]=/usr/lib/nagios/plugins/check_bacula_windows_backup

# NAGIOS-SERVER: test the Windows backup check
/usr/lib/nagios/plugins/check_nrpe -H X.X.X.118 -c check_bacula_windows_backup
```

Observed Windows result: OK – Backup-Windows completed successfully; JobId 55, incremental level I, six files, 51,160 bytes, 7 October 2026 23:15:20. The Z400 Full backup JobId 42 had previously completed successfully.

```
# NAGIOS-SERVER: append as a service in /etc/nagios4/conf.d/bacula-server.cfg
define service {
   use                 generic-service
   host_name           bacula-server
   service_description    Backup-Windows Job Status
   check_command        check_nrpe!check_bacula_windows_backup
}
```

The matching existing service uses service_description Backup-Z400 Job Status and check_nrpe! check_bacula_backup. Validate with nagios4 -v /etc/nagios4/nagios.cfg and restart Nagios only after zero errors.

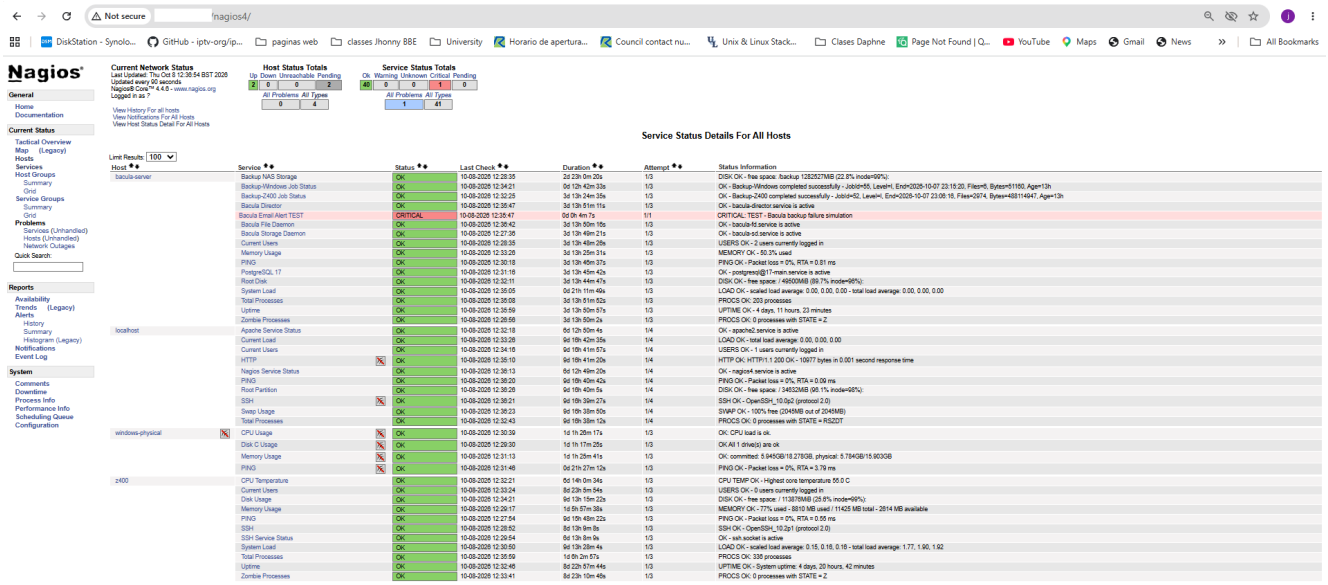


Figure 7 — Both Backup-Windows Job Status and Backup-Z400 Job Status show OK in the monitoring dashboard.

9. Performance graphing with RRDTool

An additional dark dashboard was published at http://X.X.X.83/nagios-graphs/. Eleven graphs were built: four for Z400, three for Windows and four for Bacula. Updates were scheduled approximately every ten minutes using systemd timers (bacula-rrd-update.timer and bacula-graphs.timer). Bacula scripts reside under /usr/local/bin/update-bacula-* and /usr/local/bin/generate-bacula-graphs; RRD data under /var/lib/nagios4/rrd/.

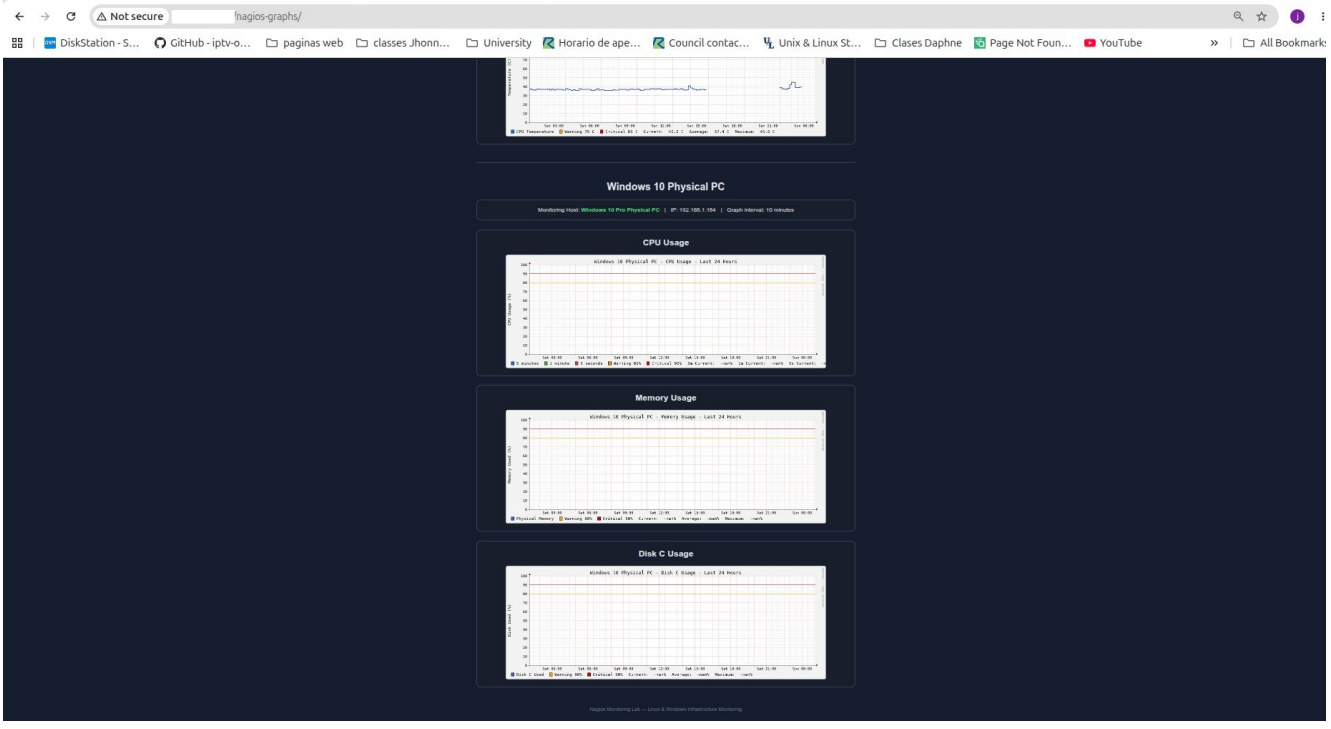


Figure 8 — Custom dark performance dashboard with RRDTool charts.

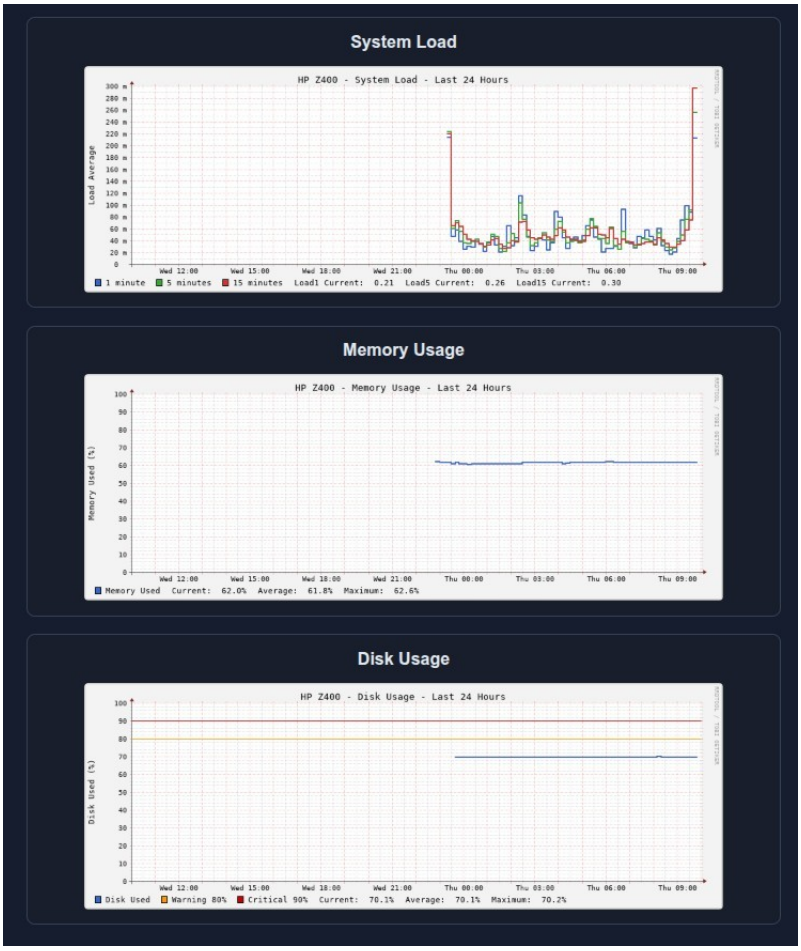


Figure 9 — Historical performance charts from the lab.

10. Email notifications via Exim4

Exim4 was configured as a Gmail SMTP smarthost, with alerts sent to a configured Yahoo mailbox. The Nagios contact is nagiosadmin, in the admins contact group. Authentication details in /etc/exim4/passwd.client must never be published. Nagios service template: notifications_enabled 1, max_check_attempts 3, check_interval 10, retry_interval 2, contact_groups admins, notification_options w,u,c,r, notification_interval 60 and notification_period 24x7.

```
# NAGIOS-SERVER
grep -E "^(enable_notifications|execute_service_checks|interval_length)=" /etc/nagios4/nagios.cfg
systemctl status exim4 --no-pager
/usr/lib/nagios/plugins/check_dummy 2 "TEST - Bacula backup failure simulation"
echo $?   # 2 = CRITICAL
```

A temporary Bacula Email Alert TEST service generated a CRITICAL notification during a controlled failure simulation. A separate System Load service generated the observed OK/RECOVERY notification. These messages demonstrate two distinct alert events, not a CRITICAL-to-RECOVERY transition for the same service. The temporary test service was removed, leaving 40 operational services.

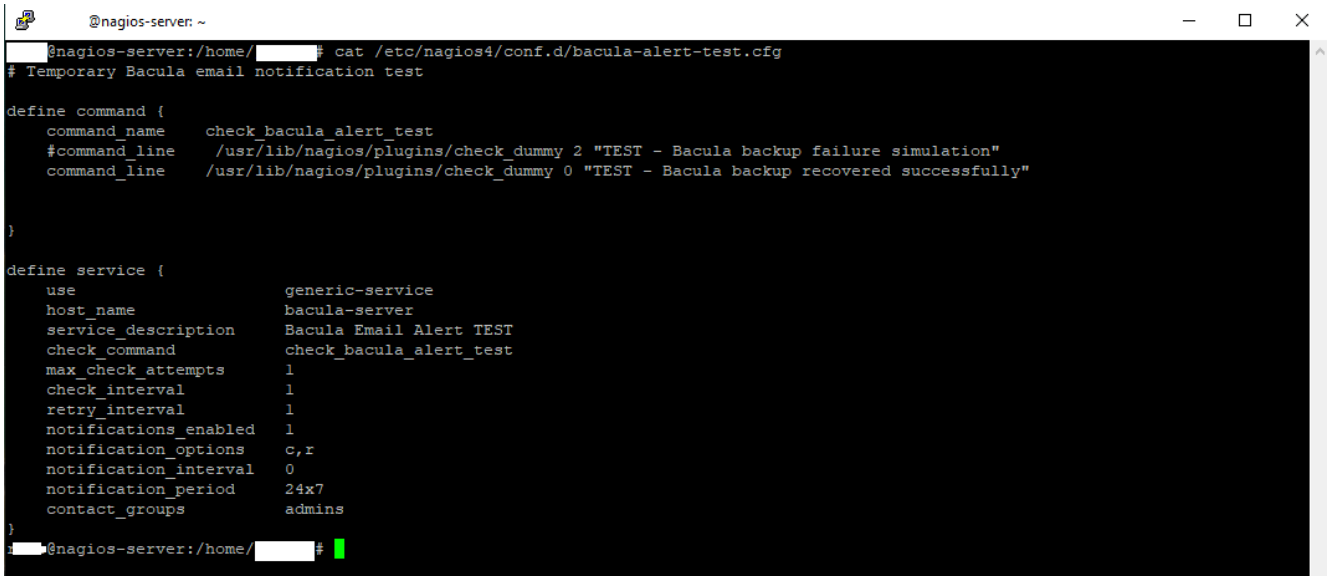


Figure 10 — Temporary test service definition, showing the CRITICAL command commented out and the OK recovery command active.

11. Validation and operational results

Test	Observed result
Nagios configuration pre-flight	0 warnings / 0 errors
Nagios UI and CGI	Working Hosts and Services pages
Z400 SSH and NRPE	OpenSSH reachable; NRPE v4.1.3
Windows NRPE checks	CPU, memory, disk and PING displayed
Bacula backup checks	Z400 and Windows job status OK
Email CRITICAL	Bacula test message received
Email RECOVERY	System Load recovery received
Temporary test removal	

12. Troubleshooting log

Issue	Resolution / lesson
Invalid max_check_attempts for z400	Corrected the host object in z400.cfg; re-ran pre-flight
CGI file downloaded	Enabled Apache CGI and corrected ScriptAlias/base URL
Duplicate check_nrpe command	Removed duplicate in nrpe_commands.cfg; used packaged check_nrpe.cfg
Over-sensitive Linux load thresholds	Adjusted for 12 logical CPUs
Windows host intermittently down	Investigated sleep and disabled notifications, not monitoring
Backup Windows plugin missing	Copied known-good Z400 plugin; changed job name; tested as nagios user
Email alert verification	Bacula CRITICAL and separate System Load RECOVERY confirmed

13. Security and production considerations

For production: use TLS certificate verification instead of verify mode = none; restrict NRPE/NSClient++ network access; keep secrets out of screenshots and source control; ensure

Windows is awake during scheduled backups; tune age thresholds to the real backup schedule; use read-only PostgreSQL permissions; monitor Exim delivery logs and queue; and document retention, escalation and acknowledgement procedures.

14. Evidence index and portfolio summary

Screenshots in this document were selected from the supplied lab capture set and show actual states at different points in the project. Earlier screenshots may show faults that were subsequently resolved. The result is a practical multi-platform infrastructure monitoring lab with automated service checks, backup health verification, historical performance visualisation and tested email notifications.

Portfolio-ready summary: Designed and implemented a Nagios Core monitoring platform on Debian, integrated Linux NRPE, Windows NSClient++ and Bacula PostgreSQL backup checks, built RRDTool performance dashboards, configured SMTP alerting, and verified a simulated Bacula CRITICAL alert alongside a separate System Load RECOVERY notification.